Oliver Krauss

Modern Code - Final Review & Code Quality

ModernCode | MTD.ba VZ SS25

Hagenberg 02.02.2026

HAGENBERG | LINZ | STEYR | WELS

FH OBERÖSTERREICH
UNIVERSITY OF APPLIED SCIENCES UPPER AUSTRIA

Final Review & Code Quality in an AI-First World

- **The Journey**: We've learned variables, control flow, methods, arrays, algorithms, and problem-solving
- **The New Reality**: Most code will be written by or with AI
- **The Critical Question**: If AI writes the code, what's YOUR role?
- **Today's Focus**: What actually matters for code quality when AI generates most code?
- **The Shift**: From writer to architect, verifier, and reviewer
- **The Goal**: Understand what to focus on, what to verify, and what AI handles

Does Code Quality Still Matter? YES Differently

The New Reality:

- AI writes most code
- You verify what you didn't write
- Understanding is MORE critical
- Verification is essential
- Architecture is your domain

What Matters Now:

- Correctness (verify it works)
- Security (AI often misses this)
- Edge cases (AI overlooks them)
- Integration (does it fit?)
- Testing (always test AI code)

Code quality principles still matter, but your role shifts from writer to verifier and architect.

The Shift: From Writer to Verifier

Your Role Changes:
Understanding code helps you verify AI output
Quality principles help you evaluate what matters

Today: What to focus on when AI writes the code

What We'll Cover Today

1. **What Matters Now**: Quality priorities in AI-first world
2. **Verification & Testing**: How to verify AI-generated code
3. **Reviewing AI Code**: What to check, what to trust
4. **Prompting for Quality**: How to ask AI for good code
5. **What Smells Matter**: Critical vs. AI-handled issues
6. **AI-First Workflows**: Iterative refinement with AI
7. **Professional Practices**: Version control, testing, collaboration with AI
8. **The New Mindset**: Verifier, architect, tester, not just coder

Focus: What you need to verify vs. what AI handles

What we need to ask

If AI writes most code, what should YOU focus on?

What we need to ask

If AI writes most code, what should YOU focus on?

- If AI can fix duplication easily, does it matter as much?
- If AI writes clean code, what's your role?
- What can AI do well? What can't it do?
- How do you verify code you didn't write?

What we need to ask

If AI writes most code, what should YOU focus on?

- If AI can fix duplication easily, does it matter as much?
- If AI writes clean code, what's your role?
- What can AI do well? What can't it do?
- How do you verify code you didn't write?

These questions shape what code quality means today

What AI Does Well

AI Excels At:

- Correct syntax
- Consistent formatting
- Code generation
- Basic refactoring
- Fixing duplication
- Following patterns

AI Struggles With:

- Edge cases
- Security vulnerabilities
- Business logic correctness
- System integration
- Architecture decisions
- Context understanding

Your focus: Verify what AI can't do well

The New Reality

Consider this scenario:

- AI generates a sorting algorithm for you
- The code looks clean and well-formatted
- Variable names are descriptive
- No obvious syntax errors

What should you check before using it?

The New Reality

Consider this scenario:

- AI generates a sorting algorithm for you
- The code looks clean and well-formatted
- Variable names are descriptive
- No obvious syntax errors

What should you check before using it?

- Does it actually work?
- What about edge cases?
- Is it secure?
- Does it fit your system?

Recap: Our Journey Through Quality

Let’s revisit what we’ve learned, viewed through the AI-first quality lens

- Each concept helps you EVALUATE code (not just write it)
- Understanding code structure helps you verify AI output
- Quality principles help you know what to check
- Today: What matters when AI writes the code?

Recap: Variables & Types - Quality Focus

What we learned:

- Variable declaration, initialization, modification
- Primitive types and their uses
- Operators and expressions

Quality principles:

- **Naming**: `playerHealth` vs `ph` - clarity matters
- **Type choices**: `int` vs `double` - precision and correctness
- **Constants**: `MAX_HEALTH` vs `100` - maintainability
- **Initialization**: Always initialize before use - prevents bugs

Focus: Understanding variables helps you verify AI-generated code is correct

Recap: Control Structures - Quality Focus

What we learned:

- `if/else`, `switch`, loops (`while`, `for`, `do-while`)
- Boolean logic and conditions
- Tracing code execution

Quality principles:

- **Complexity**: Simple conditions are easier to understand
- **Nesting**: Deep nesting increases cognitive load
- **Readability**: Clear loop conditions and meaningful variable names
- **Structure**: Consistent formatting and braces

Focus: Understanding control flow helps you trace and verify AI code logic

Recap: Methods - Quality Focus

What we learned:

- Method syntax, parameters, return types
- Design principles: SRP (Single Responsibility Principle), DRY (Don't Repeat Yourself), KISS (Keep It Simple, Stupid)
- Stepwise refinement and decomposition

Quality principles:

- **Single Responsibility**: One method, one purpose
- **Clear naming**: Verb-based names describe actions
- **Small methods**: Easier to understand, test, and maintain
- **Parameter design**: Clear inputs, meaningful outputs

Focus: A clear structure helps you maintain code longer

Recap: Arrays & Algorithms - Quality Focus

What we learned:

- Array basics, searching, sorting
- Algorithm efficiency and complexity
- Problem-solving paradigms

Quality principles:

- **Efficiency**: Choose the right algorithm for the problem
- **Clarity**: Algorithm should be understandable, not just fast
- **Correctness**: Works correctly for all cases
- **Documentation**: Complex algorithms need explanation

Key insight: Understanding algorithms helps you verify AI chose the right approach

Recap: Recursion vs Iteration - Quality Focus

What we learned:

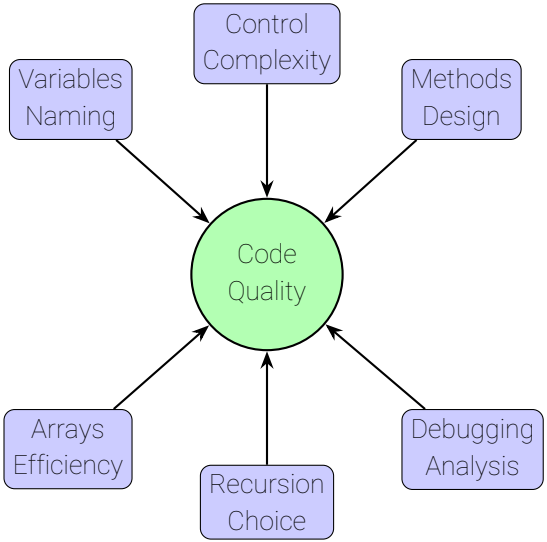
- Recursive and iterative approaches
- When to use each
- Performance trade-offs

Quality principles:

- **Appropriate choice**: Match approach to problem structure
- **Performance awareness**: Understand trade-offs
- **Clarity**: Choose the approach that's clearest for the problem
- **Maintainability**: Consider future modifications

Key insight: Understanding trade-offs helps you verify AI made appropriate choices

Everything we learned Connects to Quality



What Matters Now: The AI-First Question

If AI writes most code, what should YOU focus on?

- **The shift**: From writing code to verifying and architecting
- **The reality**: AI handles syntax, formatting, basic structure
- **Your role**: Verify correctness, security, integration, architecture
- **The question**: What quality concerns are still critical?

Not everything matters equally anymore

What Still Matters: Critical Priorities

These are MORE important when AI writes code:

1. **Correctness & Verification**
   - Does it actually work?
   - Test it thoroughly
   - Verify edge cases
2. **Security & Edge Cases**
   - AI often misses security vulnerabilities
3. **Architecture & Design**
   - AI can't design systems well
   - Integration with existing code
   - Business logic correctness
4. **Testing**
   - System tests become more important
   - Verify behavior matches requirements
5. **Understanding**
   - You need to understand to verify
   - Can you explain what it does?
   - Can you modify it if needed?

What Matters Less: AI Can Handle This

These are less critical. AI handles them well:

- **Code Duplication**
  - AI likes to duplicate code
  - Easy to refactor with AI
  - Less of a concern for code quality
- **Syntax Perfection**
  - AI generates correct syntax
  - Compiler catches syntax errors
  - Focus on logic, not syntax
- **Formatting & Style**
  - AI is consistent with formatting
  - Auto-formatters handle this
  - Not worth manual effort
- **Basic Refactoring**
  - AI can extract methods, rename variables
  - Ask AI to refactor when needed
  - Focus on higher-level concerns

Don't optimize what AI already optimizes

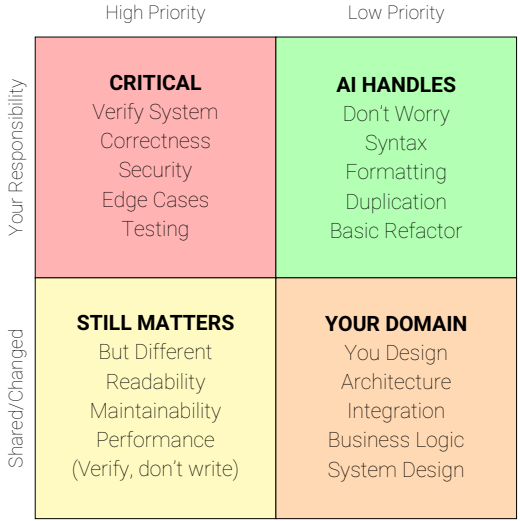
What Matters Differently: Changed Priorities

These still matter, but the focus shifts:

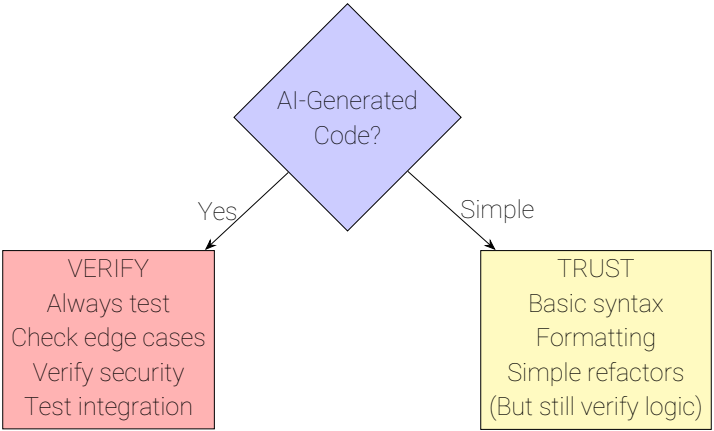
- **Readability**
  - Old: "Can humans read it?"
  - New: "Can AI understand it for modifications?"
  - Still important, but different perspective
- **Maintainability**
  - Old: "Can humans modify it?"
  - New: "Can AI modify it safely?"
  - Structure matters for AI-assisted changes
- **Performance**
  - Still matters for critical paths
  - Focus on verifying optimization, not doing it

The principles remain, but application changes

Quality Priority Matrix: What to Focus On



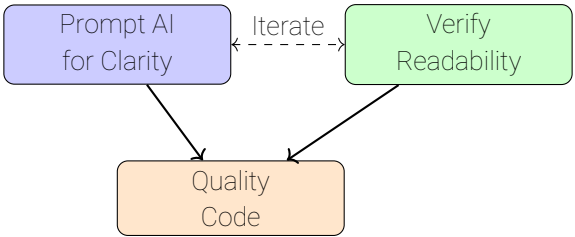
Decision Framework: When to Verify, When to Trust



Principle: Trust AI for syntax and structure, but ALWAYS verify logic and correctness

Readability & Maintainability: AI-First Perspective

- **Readability**: Can AI understand it for modifications?
- **Maintainability**: Can AI modify it safely?
- **Your role**: Verify clarity, prompt for improvements
- **Focus**: Evaluate and prompt, not write from scratch



Naming: How to Prompt AI for Good Names

Why naming still matters:

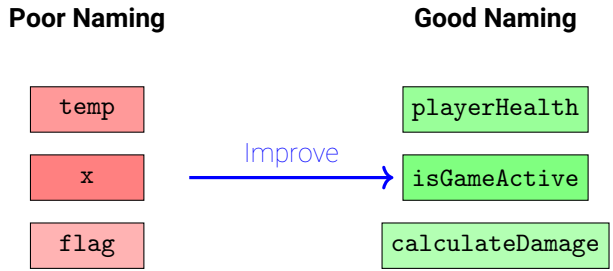
- You need to understand code to verify it
- Good names help you evaluate correctness
- AI needs the same thing for understanding code

How to prompt AI:

- "Use descriptive variable names like `playerHealth` not `ph`"
- "Name methods with verb-noun pattern: `calculateDamage()`"
- "Use boolean names with `is/has/can` prefix"
- "Extract magic numbers into named constants"

Your role: Verify names are clear and meaningful

Naming Quality: Visual Comparison



Unclear → Clear

Good names are self-documenting and reduce cognitive load

Naming Patterns for Different Contexts

Context	Pattern	Example
Methods	Verb + Noun	`calculateDamage()`
Variables	Noun/Adjective	`playerHealth`, `isAlive`
Constants	UPPER_SNAKE_CASE	`MAX_HEALTH`
Booleans	`is/has/can` prefix	`isGameActive`
Collections	Plural noun	`enemies`, `scores`
Counters	`i, j, k` (in loops)	`for (int i = 0; ...)`

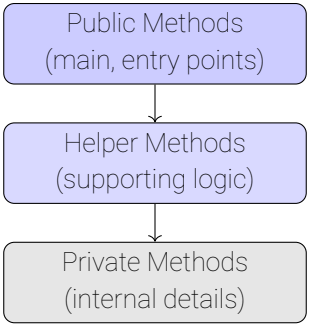
Domain-specific naming:

- Use terms from the problem domain
- `enemyHealth` is better than `value1`
- `calculateDamage()` is better than `compute()`

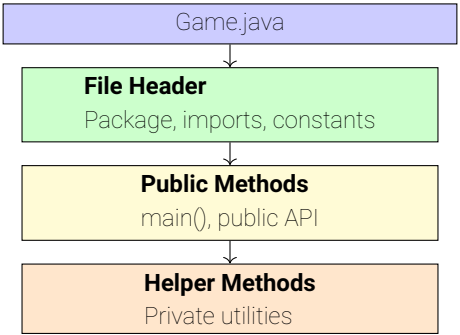
Code Organization: Method Ordering

Principles:

- **Public before private**: Most important methods first
- **Logical grouping**: Related methods together
- **Call order**: Called methods before calling methods (when possible)
- **Consistency**: Same organization throughout file



Code Organization: File Structure



Consistent organization makes code easier to navigate and understand

Comments & Documentation: When to Comment

When to Comment:

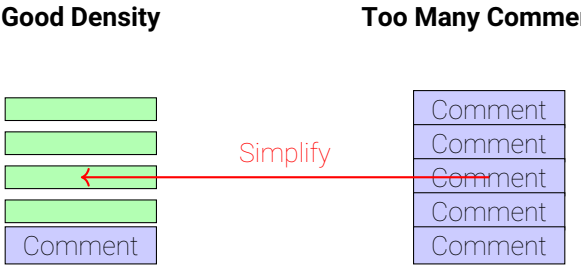
- **Why**, not **what**
- Complex algorithms
- Non-obvious decisions
- Business rules
- API documentation

When NOT to Comment:

- Obvious code
- Redundant explanations
- Outdated comments
- Commented-out code
- Self-documenting code

Principle: Code should be self-documenting. Comments explain why, not what.

Comment Density Visualization



Green = Code, Blue = CommentsToo many comments suggest unclear code

Self-Documenting Code

With Comments:

```
// Check if player is dead
if (hp <= 0) {
    // End the game
    endGame();
}
```

Self-Documenting:

```
if (isPlayerDead(playerHealth)) {
    endGame();
}
```

Better: Extract method with clear name instead of comment

```
static boolean isPlayerDead(int health) {
    return health <= 0;
}
```

Formatting & Style: Consistency Matters

Why formatting matters:

- Inconsistent formatting is distracting
- Good formatting makes structure visible
- Team consistency enables collaboration

Inconsistent:

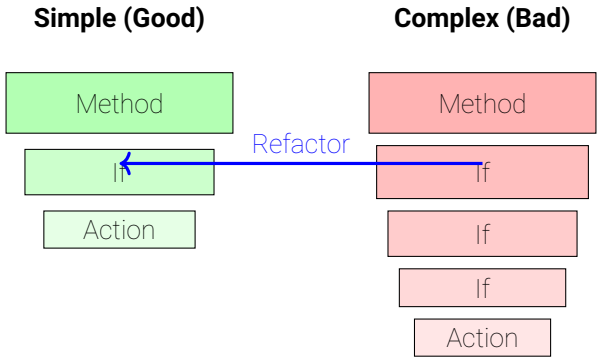
```
if(x>0){
result=x*2;
}else{
result=0;
}
```

Consistent:

```
if (x > 0) {
    result = x * 2;
} else {
    result = 0;
}
```

Key: Use IDE auto-formatting and style guides

Complexity Management: Visualizing Nested Complexity



Deep nesting increases cognitive load. Extract methods to reduce complexity.

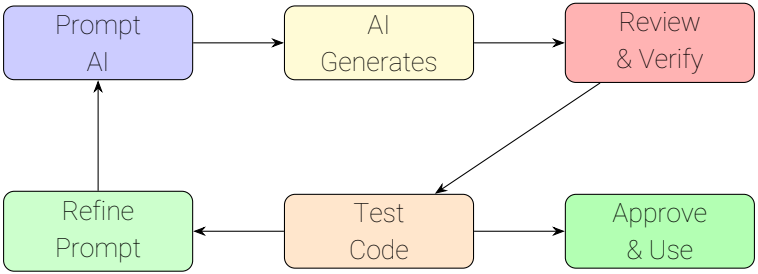
Reviewing AI-Generated Code: The Critical Skill

The new reality: Most code is/will be AI-generated, so review becomes essential

- **Purpose**: Verify correctness, find AI mistakes, ensure security
- **Process**: AI generates → You review → Test → Iterate
- **Critical**: AI can be wrong, miss edge cases, introduce vulnerabilities
- **Your role**: Verifier, tester, architect, not just reviewer

IMPORTANT:: Never trust AI code blindly! Always verify

AI Code Review Workflow



AI-first workflow: Prompt → Generate → Review → Test → Iterate

AI Code Review Checklist: What to Check

CRITICAL: Always Verify

- ☐ **Correctness**: Does it actually work?
- ☐ **Edge Cases**: Empty inputs, null values, boundaries
- ☐ **Security**: Input validation, injection risks
- ☐ **Testing**: Write and run tests
- ☐ **Integration**: Does it fit your system?

VERIFY: Business Logic

- ☐ Matches requirements?
- ☐ Handles errors correctly?
- ☐ Returns expected outputs?

CHECK: AI Common Mistakes

- ☐ Hallucinated APIs (made-up methods)
- ☐ Wrong algorithm choice
- ☐ Over-engineering (unnecessary complexity)
- ☐ Missing edge cases
- ☐ Security vulnerabilities

SKIP: AI Handles Well

- ☐ Syntax (usually correct)
- ☐ Formatting (consistent)
- ☐ Basic structure (good)

Common AI Code Mistakes to Watch For

Hallucinations:

- Made-up API methods
- Non-existent libraries
- Wrong method signatures

Missing Context:

- Missing edge cases
- Wrong algorithm choice
- Doesn't fit your system

Security Blind Spots:

- Missing input validation
- SQL injection risks
- Missing authentication

Integration Issues:

- Wrong data types
- Missing dependencies
- Incompatible interfaces

Always verify these! AI gets them wrong frequently

AI Can Review Code Too

AI code review capabilities:

- **Analyzes code**: Can examine code structure, logic, and style
- **Identifies issues**: Finds bugs, code smells, and potential problems
- **Suggests improvements**: Provides refactoring recommendations
- **Checks patterns**: Verifies adherence to coding standards

Use AI review as a first pass, but always verify with human judgment

- AI catches many issues quickly
- Human reviewer adds context and domain knowledge
- Best practice: AI review → Human verification

AI Review Output Example

AI Code Review Results:

- Bug: Potential null pointer at line 15
- Style: Method name unclear (line 8)
- Complexity: Method too long (45 lines)
- Good: Clear variable names

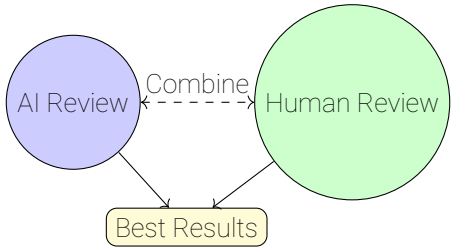
Suggestions:

1. Add null check before accessing array
2. Rename `calc()` to `calculateDamage()`
3. Extract logic into smaller methods

AI provides structured feedback, but human reviewer adds context and judgment

AI Review Limitations

- **Context**: May miss domain-specific issues
- **False positives**: Can flag non-issues
- **Design judgment**: Can't evaluate architectural decisions well
- **Business logic**: Doesn't understand requirements
- **Verification needed**: Always verify AI suggestions



Best practice: Use AI for initial screening, human for final judgment

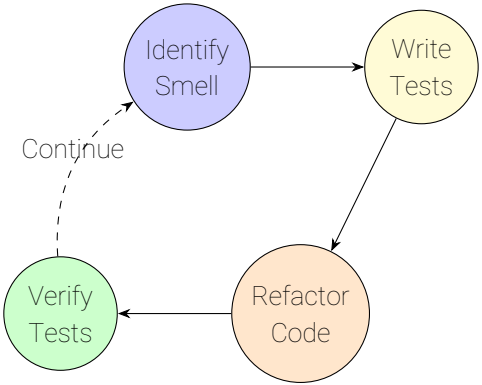
Refactoring in AI-First World

The shift: From manual refactoring to prompting AI to refactor

- **AI can refactor**: Extract methods, rename, simplify. AI does this well
- **Your role**: Identify when refactoring is needed, prompt AI to do it
- **Decision**: When to refactor vs. when to regenerate code
- **Verification**: Always test after AI refactoring

Key insight: Understand refactoring patterns to know when to ask AI for them

Refactoring Cycle



Safety net: Tests ensure behavior doesn't change during refactoring

Refactoring vs. Regeneration: When do What

Prompt AI to Refactor:

- Code works but needs cleanup
- Small structural improvements
- Extract methods, rename
- Simplify logic
- Tests exist to verify

Example: "Refactor this method to extract the calculation into a separate method"

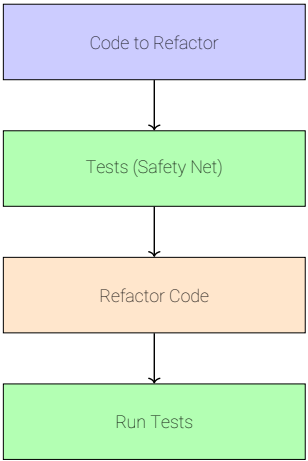
Regenerate from Scratch:

- Fundamental design flaw
- Too many issues to fix
- Major logic errors
- Better to start fresh
- Requirements changed significantly

Example: "Rewrite this with a different algorithm approach"

Decision: Can AI improve it incrementally? If yes, refactor. If no, regenerate.

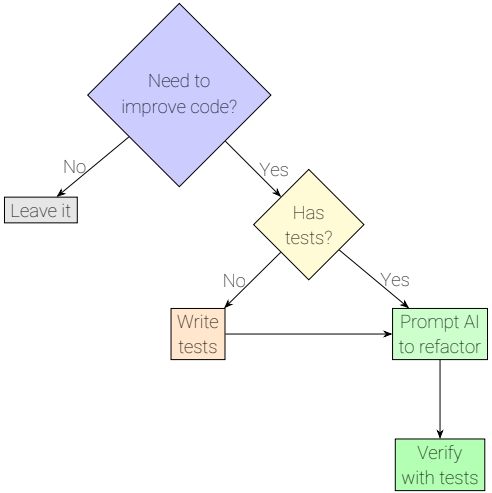
Refactoring Safety: The Safety Net



If tests pass:
Refactoring successful
If tests fail:
Behavior changed
Fix and retry

Tests ensure behavior doesn't change during refactoring

Refactoring Decision Tree



Decision: If you need to improve code, ensure tests exist first

What are Code Smells?

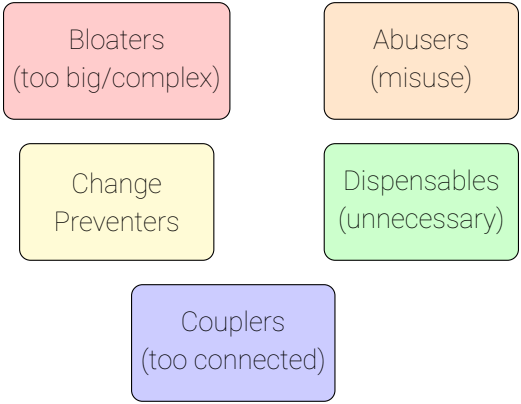
Definition: Warning signs that code might have problems

- **Not errors**: Code works, but something feels wrong
- **Warning signs**: Suggest deeper problems or future issues
- **Subjective**: Context-dependent, not absolute rules
- **Actionable**: Can be fixed through refactoring

Analogy: Like a bad smell in a room. Nothings broken, but it suggests a problem

- Examples: Long methods, duplicate code, unclear names, magic numbers
- In AI-first world: Some smells matter more, some less

Code Smell Categories



Common categories of code smells (but not all matter equally with AI)

What Code Smells Matter Now?

The shift: Not all smells are equal. Some matter more with AI

- **Critical smells**: AI often creates these.
- **Moderate smells**: AI can fix, but worth verifying
- **Low priority**: AI handles these well. Don't worry

Key insight: Focus on smells that indicate correctness, security, or integration issues

Code Smell: Long Method

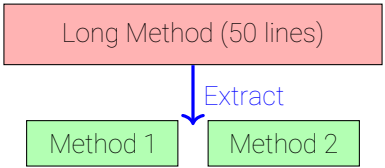
Problem: Method does too many things, hard to understand

Smell Indicators:

- Method > 20-30 lines
- Multiple responsibilities
- Hard to name clearly
- Needs many comments

Refactoring:

- Extract Method
- Break into smaller methods
- Each method does one thing
- Clear, descriptive names



Code Smell: Duplicate Code

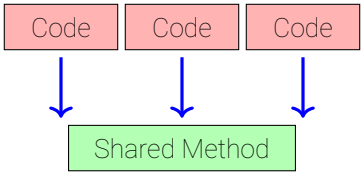
Problem: Same code appears in multiple places

Smell Indicators:

- Copy-paste code
- Similar logic repeated
- Changes need multiple edits
- Inconsistency risk

Refactoring:

- Extract Method
- Create shared utility
- Single source of truth
- DRY principle



Code Smell: Magic Numbers

Problem: Numbers without meaning in code

Smell Indicators:

- Literal numbers in code
- Unclear meaning
- Hard to change
- No explanation

Before:

```
1    if (hp > 0 && hp < 100) {
2        heal(10);
3    }
```

Refactoring:

- Extract Constant
- Named constants
- Clear meaning
- Easy to modify

After:

```
1    static final int MIN_HEALTH = 0;
2    static final int MAX_HEALTH = 100;
3    static final int HEAL_AMOUNT = 10;
4    if (hp > MIN_HEALTH &&
5        hp < MAX_HEALTH) {
6        heal(HEAL_AMOUNT);
7    }
```

Code Smell: Dead Code

Problem: Code that's never executed

Smell Indicators:

- Unused methods
- Commented-out code
- Unreachable code
- Unused variables

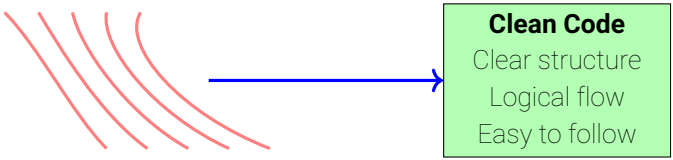
Refactoring:

- Delete unused code
- Remove comments
- Clean up
- Use version control

Principle: Dead code adds confusion. If you need it later, use version control.

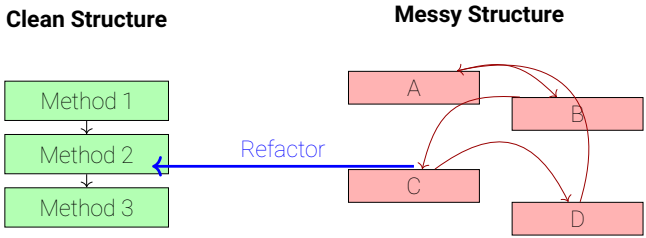
Anti-pattern: Spaghetti Code

Problem: Tangled, unstructured code with unclear flow



Solution: Structure code logically, extract methods, reduce complexity

Code Structure: Clean vs Messy



Clean structure: Clear flow, logical organization. Messy: Tangled, hard to follow.

Code Smells: What Matters with AI?

Which code smells are more/less important when AI writes code?

LESS Important (AI Handles):

- **Duplicate Code**: AI can refactor easily.
- **Magic Numbers**: AI can extract to constants. Low priority.
- **Dead Code**: AI rarely generates unused code.

MODERATE (AI Can Fix):

- **Long Method**: AI can extract methods.
- **Spaghetti Code**: AI usually structures well.

MORE Important (You Must Check):

- **Security Smells**: AI often skips security.
- **Edge Cases**: AI frequently overlooks boundary conditions.
- **Integration Problems**: AI may not understand your system.

Key Insight:

- Structural smells (duplication, formatting) → AI handles
- Logic smells → You must verify

Principle: Focus on smells that indicate bugs or security issues, not style issues.

Understanding Algorithms to Verify AI Code

Why this matters in AI-first world:

- AI generates algorithms. You need to verify they're appropriate
- Understanding algorithms helps you evaluate AI's choices
- You can prompt AI for optimizations when needed
- Focus: Verify optimization, not implement it yourself

Key insight: You need to understand algorithms to verify AI chose the right approach

- We'll look at optimization techniques AI might use
- Your role: Understand and verify, not implement from scratch

Algorithm Optimization: When and Why

What is optimization?

- Improving algorithm efficiency (time/space)
- Making code faster or using less memory
- Trading one resource for another

When to optimize:

- Performance is critical
- Code is too slow
- Memory usage is high
- After profiling identifies bottlenecks

When NOT to optimize:

- Prematurely (before measuring)
- Code works fine
- Readability suffers too much
- Not a bottleneck

Principle: Measure first, optimize second. AI can help with both.

Memoization (Caching)

Problem: Recursive Fibonacci recalculates same values repeatedly

Without Memoization:

- `fib(5)` calls `fib(4)` and `fib(3)`
- `fib(4)` calls `fib(3)` and `fib(2)`
- `fib(3)` calculated multiple times!
- Exponential time: $O(2^n)$

With Memoization:

- Store results in cache
- Check cache before calculating
- Reuse stored results
- Linear time: $O(n)$

Trade-off: Space for time → use memory to save computation

Memoization: Fibonacci Example

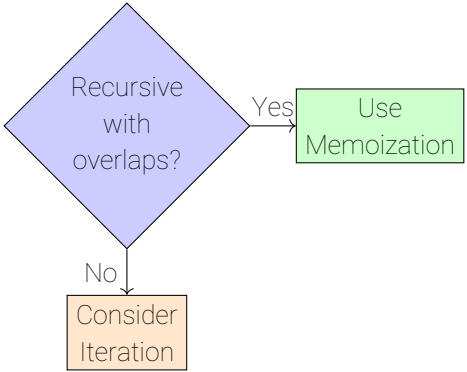
Implementation with cache:

```
1     static int[] cache = new int[100];
2
3     static int fibonacciMemoized(int n) {
4         // Base case
5         if (n <= 1) {
6             return 1;
7         }
8         // Check cache
9         if (cache[n] != 0) {
10            return cache[n];  // Cache hit!
11        }
12        // Calculate and store
13        cache[n] = fibonacciMemoized(n - 1) +
14                   fibonacciMemoized(n - 2);
15        return cache[n];
16    }
```

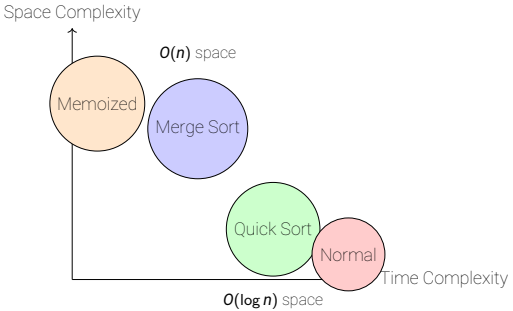
Performance: $O(n)$ time, $O(n)$ space (vs $O(2^n)$ without memoization)

When to Use Memoization

- **Overlapping subproblems**: Same calculations repeated
- **Expensive computations**: Worth storing results
- **Recursive problems**: Natural fit for memoization
- **Space available**: Can afford memory for cache



Space-Time Trade-offs



Space-Time Trade-offs: Examples

Algorithm	Time	Space
Merge Sort	$O(n \log n)$	$O(n)$ (needs extra array)
Quick Sort	$O(n \log n)$	$O(\log n)$ (in-place)
Fibonacci (recursive)	$O(2^n)$	$O(n)$ (stack)
Fibonacci (memoized)	$O(n)$	$O(n)$ (cache)
Fibonacci (iterative)	$O(n)$	$O(1)$ (no cache)

When to prioritize:

- **Time**: Real-time systems, user-facing applications
- **Space**: Embedded systems, memory-constrained devices
- **Balance**: Most applications need both

Algorithm Optimization Techniques

Early Termination: Stop as soon as answer is found

```
// Linear search with early exit
static int findIndex(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;  // Early exit!
        }
    }
    return -1;
}
```

Loop Optimization: Reduce iterations when possible

```
// Instead of: for (int i = 0; i < arr.length; i++)
// Use: for (int i = 0; i < arr.length - 1; i++)  // If last element handled separately
```

Key: Optimize only when it matters. Profile first!

When Optimization Helps vs When It Doesn't

Optimize When:

- Profiler shows bottleneck
- Algorithm is inefficient
- Large data sets
- Performance critical
- Clear improvement path

Don't Optimize When:

- Premature (no profiling)
- Micro-optimizations
- Hurts readability
- No measurable impact
- Wrong algorithm choice

Principle: Measure first, optimize second. Readability usually matters more.

When to Optimize

Should you optimize everything?

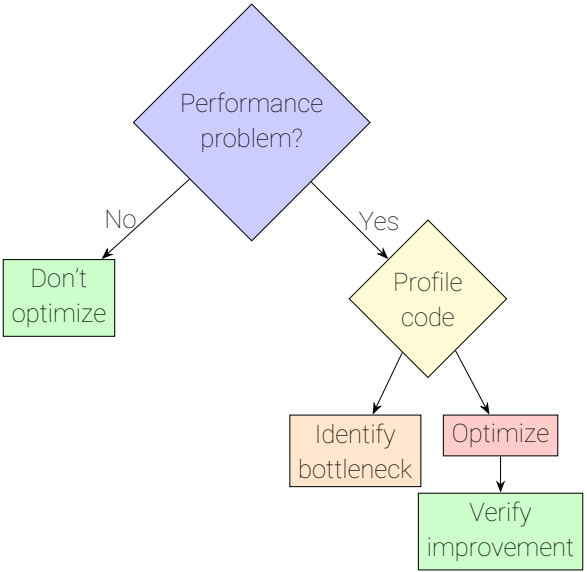
When to Optimize

Should you optimize everything?

- **Answer**: No! Optimize only what matters
- **Principle**: Premature optimization is the root of all evil
- **Process**: Measure → Identify → Optimize → Verify

Key insight: Most code doesn't need optimization. Focus on bottlenecks.

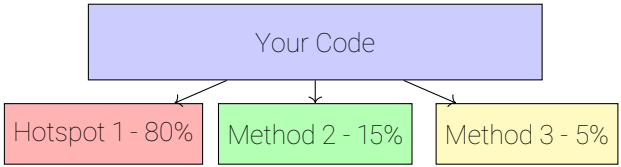
Optimization Decision Tree



Profiling First: Finding Bottlenecks

What is profiling? Measuring where code spends time

- **Profiler tools**: Show which methods take most time
- **Hotspots**: Code sections that execute frequently
- **Call graphs**: Visualize method call relationships
- **Memory profiling**: Track memory usage



Optimize hotspot 1!Don't waste time on method 3

Optimization Techniques

Algorithm Selection: Choose the right algorithm

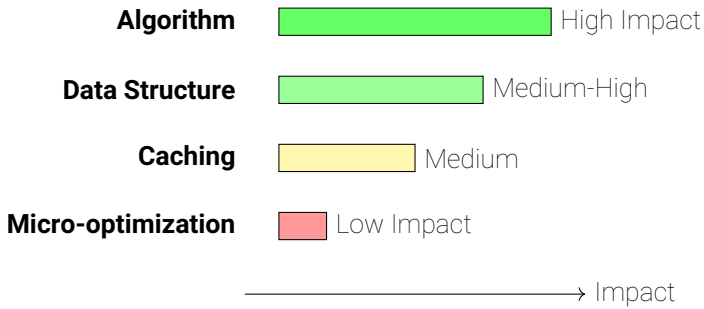
- **Search**: Binary search ($O(\log n)$) vs Linear search ($O(n)$)
- **Sort**: Quick Sort vs Bubble Sort
- **Data structures**: Array vs List vs Set (depends on operations)

Data Structure Choice: Match structure to operations

- **Frequent access**: Array (fast indexing)
- **Frequent insertions**: List (dynamic size)
- **Lookups**: Set/HashMap (fast search)

Key: Right algorithm/structure often matters more than micro-optimizations

Optimization Impact Visualization



Focus on high-impact optimizations first

Optimization Pitfalls

Premature Optimization: Optimizing before measuring

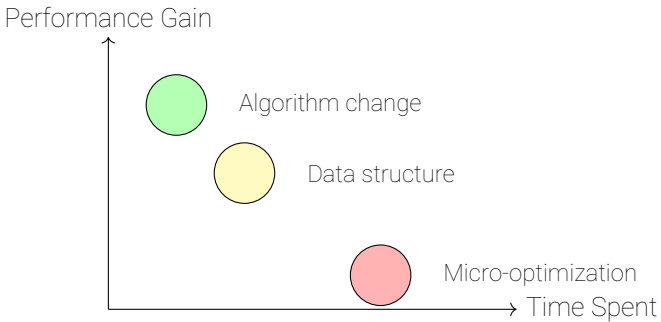
- **Problem**: Optimizing code that's not a bottleneck
- **Waste**: Time spent on code that doesn't matter
- **Risk**: May hurt readability for no benefit

Micro-optimizations: Tiny improvements that don't matter

- **Example**: Optimizing loop variable increment
- **Reality**: Compiler already optimizes this
- **Better**: Focus on algorithm choice

Principle: Optimize only what profiling shows is slow

Optimization ROI (Return on Investment)

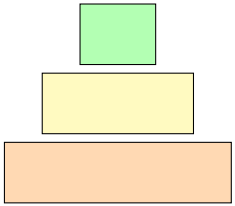


Best ROI: Algorithm and data structure choices

Worst ROI: Micro-optimizations

Testing Practices

Testing Pyramid:



Unit Tests (many)

Integration Tests (some)

System Tests (few)

Principles:

- **Many unit tests**: Fast, isolated, test individual methods
- **Some integration tests**: Test component interactions
- **Few end-to-end tests**: Test complete workflows

Writing Testable Code

Characteristics of testable code:

- **Small methods**: Easy to test in isolation
- **Clear inputs/outputs**: Predictable behavior
- **No hidden dependencies**: Dependencies are explicit
- **Pure functions**: Same input → same output

Example:

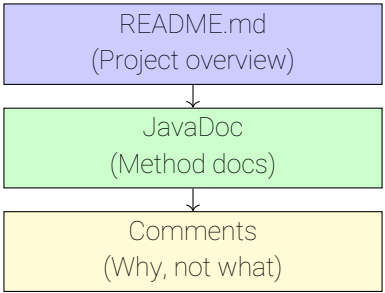
```
// Testable: Clear input, clear output
static int calculateDamage(int base, int multiplier) {
    return base * multiplier;
}

// Hard to test: Hidden dependency, side effects
static void attack() {
    // Uses global state, prints, modifies files...
}
```

Code Documentation

Types of documentation:

- **README**: Project overview, setup instructions
- **API docs**: Method signatures, parameters, return values
- **Comments**: Explain why, not what
- **Code examples**: Show how to use code

Documentation structure:



Using AI for Code Quality Tasks

AI can help with multiple code quality tasks:

- **Code Review**: Already covered before (reviewing AI-generated code)
- **Refactoring**: Prompt AI to improve code structure
- **Code Analysis**: Ask AI to analyze complexity, performance, smells
- **Optimization**: Request AI suggestions for performance improvements

Key principle: AI assists with quality tasks, but you verify and decide

AI for Refactoring

How to use AI for refactoring:

- **Ask for refactoring**: "Refactor this code to improve readability"
- **Specific patterns**: "Extract this method" or "Simplify this conditional"
- **Code smells**: "Fix code smells in this method"
- **Step-by-step**: Ask for one refactoring at a time

Example prompt:

```
"Refactor this method to extract the damage calculation
into a separate method with a clear name."
```

Always verify: Test that refactored code works the same

AI Refactoring Suggestions

AI Refactoring Suggestions:

- Extract Method: Lines 5-10 can be extracted to `calculateDamage()`
- Rename Variable: `temp` → `totalDamage`
- Simplify Conditional: Complex if can use helper method
- Consider: Add constants for magic numbers

Refactored Code:

```
int damage = calculateDamage(base, multiplier);
int totalDamage = damage + bonus;
if (canAttack()) { attack(); }
```

AI suggests improvements, but you decide what to apply

AI for Code Analysis

Complexity analysis:

- "Analyze the time complexity of this algorithm"
- "What is the space complexity?"
- "Suggest optimizations for this code"

Performance suggestions:

- "Identify performance bottlenecks in this code"
- "Suggest faster algorithms for this problem"
- "Optimize this method for speed"

Code smell detection:

- "Find code smells in this class"
- "Identify anti-patterns in this code"
- "Suggest refactoring opportunities"

AI Analysis Output Example

Code Analysis Results:

Complexity:
- Time: $O(n^2)$ - nested loops
- Space: $O(1)$ - constant space

Performance:
- Bottleneck: Inner loop (line 8)
- Suggestion: Use HashMap for $O(1)$ lookup

Code Smells:
- Long Method (45 lines)
- Magic Numbers (lines 12, 15)
- Duplicate Code (lines 8-10, 18-20)

Recommendations:
1. Extract method for inner loop logic
2. Replace magic numbers with constants
3. Consider algorithm optimization

AI Limitations & Best Practices

Limitations:

- **Context**: May miss domain-specific issues
- **False positives**: Can flag non-issues
- **Design judgment**: Can't evaluate architecture well
- **Business logic**: Doesn't understand requirements

Best practices for using AI:

- **Verify suggestions**: Always test AI recommendations
- **Use as assistant**: AI helps, human decides
- **Be specific in prompts**: Clear requests get better results
- **Iterate**: Refine prompts based on AI output

Principle: AI is a powerful tool, but human judgment is essential

AI-First Workflow - From Prompt to Production

Goal: Work with AI to produce and verify quality code

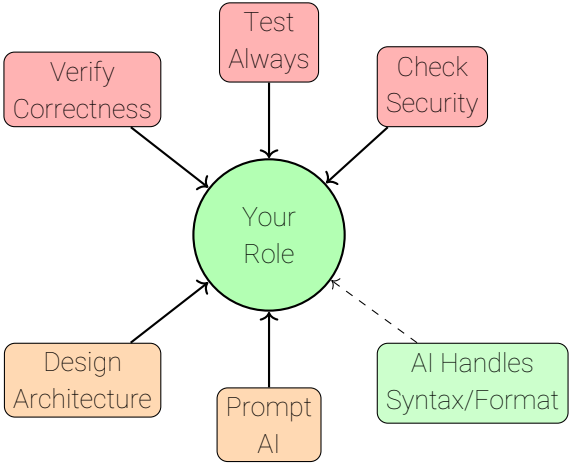
Scenario: Need a sorting algorithm for your application

AI-First Process:

1. Prompt AI for initial code
2. Review AI output (what to check)
3. Identify issues (edge cases, security, etc.)
4. Test the code thoroughly
5. Iterative refinement with AI
6. Final verification

Lesson: Your role is verification and iteration, not writing from scratch

The New Code Quality Mindset



Focus on verification and architecture. AI handles implementation

Code Quality Checklist

Readability

- ☐ Clear variable names
- ☐ Well-structured code
- ☐ Consistent formatting
- ☐ Appropriate comments

Maintainability

- ☐ Small methods
- ☐ Single responsibility
- ☐ No code smells
- ☐ Easy to modify

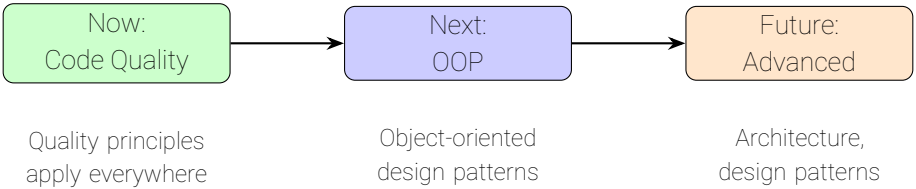
Correctness

- ☐ Works correctly
- ☐ Handles edge cases
- ☐ Error handling
- ☐ Tested

Performance

- ☐ Appropriate algorithm
- ☐ Efficient enough
- ☐ No premature optimization

Your Journey Forward



What's next:

- Object-Oriented Programming (next semester)
- Design patterns and architecture
- Advanced algorithms and data structures
- Software engineering practices

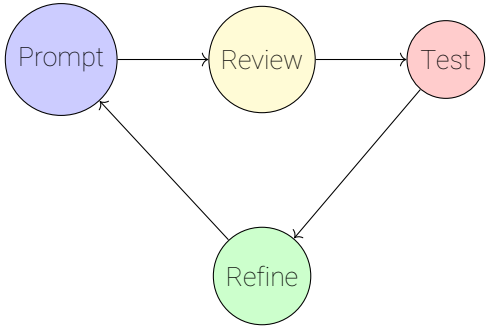
Resources for Continued Learning

- **Books**: "Clean Code" by Robert Martin, "Refactoring" by Martin Fowler
- **Practice**: Code review exercises, refactoring challenges
- **Tools**: Linters, formatters, profilers
- **Communities**: Code review platforms, open source projects
- **AI**: Use AI assistants for learning and practice

Remember: Code quality is a journey, not a destination. Keep learning and improving.

Final Thoughts: The New Code Quality Mindset

- **Your role shifts**: From writer to verifier, architect, tester
- **Focus on what matters**: Correctness, security, architecture, integration
- **Let AI handle**: Syntax, formatting, basic refactoring, duplication
- **Always verify**: Test AI code, check edge cases, verify security
- **Iterate with AI**: Prompt → Review → Test → Refine



Thank You!

Have fun during the semester break

In an AI-first world, your role is to verify, architect, and test. Not just to write code.

Focus on what matters: correctness, security, and integration. Let AI handle the rest.

Keep verifying, keep testing, keep improving!